SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

ANSWERS

1. Reverse Engineering an ATM Withdrawal Module

1.1 Objective and inferred behaviour

The objective is to infer the ATM withdrawal rules from observable inputs and outputs, then reconstruct a black-box test suite. Since the exact ATM specification is not supplied, assume for illustration that the per-transaction withdrawal limit is ₹25,000, withdrawals must be positive multiples of ₹100, and the account must retain a minimum balance of ₹0. These values are assumptions, not facts from the question.

1.2 Functional decomposition

Component	Responsibility	Typical failure to test
Card/PIN validation	Checks card status and PIN	Blocked card or incorrect PIN accepted
Account selection	Confirms selected account exists	Invalid account accepted
Amount validation	Checks numeric, positive, denomination and limit	Zero, negative, decimal or over-limit amount accepted
Balance validation	Checks available balance and holds	Withdrawal exceeds available funds
Cash dispenser	Dispenses requested notes	Wrong amount or no cash despite debit
Ledger update	Debits account exactly once	Duplicate debit or missing debit
Receipt / message	Displays result and remaining balance	Misleading success/error message

1.3 Equivalence classes and boundary values

Input / rule	Valid class	Invalid classes	Boundary tests (illustrative)
Withdrawal amount	₹100–₹25,000; positive multiple of ₹100	Zero, negative, non-numeric, decimal, non-multiple of ₹100, above limit	₹0, ₹100, ₹200, ₹24,900, ₹25,000, ₹25,100
Available balance	Amount ≤ available balance	Amount greater than balance; balance unavailable/negative	Balance ₹4,999 / ₹5,000 / ₹5,001 for a ₹5,000 request
PIN/card	Active card and correct PIN	Wrong PIN, blocked/expired card	Last allowed wrong attempt; one further attempt
Network/device	ATM online and cash available	Offline, cash empty, host timeout	Failure before debit; timeout after debit

1.4 Reconstructed test cases

TC	Precondition / input	Expected result	Technique
ATM01	Valid card/PIN; balance ₹10,000; withdraw ₹1,000	₹1,000 dispensed; balance ₹9,000; one debit	Positive / ECP
ATM02	Withdraw ₹0	Rejected; no debit	Invalid class / BVA
ATM03	Withdraw −₹100	Rejected; no debit	Invalid class
ATM04	Withdraw ₹100	Accepted if balance/cash sufficient	Lower boundary
ATM05	Withdraw ₹25,000	Accepted only if all other rules permit	Upper boundary
ATM06	Withdraw ₹25,100	Rejected; no debit	Above upper boundary
ATM07	Withdraw ₹150	Rejected if denomination rule is ₹100 multiples	ECP
ATM08	Withdraw ₹10,001 with balance ₹10,000	Insufficient-funds message; no debit/cash	Balance boundary
ATM09	Wrong PIN	Access denied; failure counter updated	Negative test
ATM10	Blocked/expired card	Transaction denied	ECP
ATM11	ATM has insufficient cash	No debit; clear message; card handled safely	Exception test

1.5 Likely original technique and adequacy

The observed examples in the question suggest Equivalence Class Partitioning (valid/invalid amounts and account states) and Boundary Value Analysis (withdrawal limit and balance edges). Error-message checks may also indicate negative testing. These techniques are suitable for input validation, but they are not adequate alone: they do not prove correct ledger updates, cash-dispensing coordination, timeout recovery, or duplicate-request prevention.

Conclusion: combine ECP and BVA with decision-table testing for combinations of amount, balance, card status and ATM cash; add state/exception tests for timeout, reversal and device failure. Verify both the visible response and the account/cash side effects.

2. Reverse Engineering White-Box Testing from Execution Logic

The assessment refers to code snippets and execution traces, but none are included in the supplied question sheet. The following is a transparent illustrative reconstruction using a small transaction-validation module. Replace the conditions if your instructor has supplied a separate code snippet.

2.1 Decisions, loops and paths

Path	Condition sequence	Expected outcome
P1	D1 = False	Reject invalid amount
P2	D1 = True, D2 = False	Insufficient funds
P3	D1 = True, D2 = True, D3 = False	Lock account
P4	D1 = True, D2 = True, D3 = True	Debit and success

2.3 Coverage evaluation and improved strategy

Statement coverage: every executable statement runs at least once; this does not guarantee that every decision outcome has been tested.
Branch/decision coverage: exercise both True and False outcomes of every decision.
Condition coverage: for compound conditions, make each atomic condition True and False at least once.
Path coverage: execute each feasible independent path; loops require representative zero-iteration, one-iteration and repeated-iteration tests.
Use assertions on output messages, balance changes, error handling and absence of side effects.
Adequacy cannot be calculated from the original submission because no actual traces or tests were supplied.
A coverage report should list executed statements, branches, conditions and feasible paths after running the real program.

3. Login and Transaction System: Inferring the Testing Approach

3.1 Component decomposition

Component	Key test concerns
Login UI/API	Empty fields, malformed input, valid/invalid credentials, error consistency
Authentication service	Password verification, account lockout, MFA, session creation
Session/access control	Expired session, logout, privilege enforcement, token replay
Transaction validation	Amount, account status, limits, available balance
Transaction processing	Debit/credit consistency, duplicate submission, rollback
Audit and notification	Accurate logs, privacy of messages, correct notifications

3.2 Likely techniques inferred from defect patterns

Defect pattern in logs (if observed)	Likely missed technique / interpretation
Failures at empty or malformed fields	ECP and negative input testing may be incomplete
Failures only at minimum/maximum amount	BVA may be missing or incorrectly selected
Valid login but unauthorized transaction	Access-control, state or path tests are insufficient
Duplicate debit after retry	Retry/idempotency and exception-path testing is missing
Incorrect response for combinations of account status and amount	Decision-table testing may be inadequate
Defects in branches not reached by normal tests	Branch/path coverage is insufficient

These are diagnostic interpretations, not claims about actual defects: the prompt says logs exist but does not include their entries.

3.3 Integrated test strategy

Test area	Representative tests	Coverage target
Login input	Blank, malformed, valid, invalid credentials	ECP + BVA
Lockout	One attempt below limit, at limit, one above limit	BVA + state transition
Session	Valid, expired, logged-out and tampered session	State/access-control
Transaction amount	Zero, minimum, maximum, over-limit, malformed	ECP + BVA
Balance	Amount below, equal to and above available balance	BVA
Transaction state	Pending → Success; Pending → Reversal; Pending → Timeout	State-transition testing
Control flow	Success, validation failure, insufficient funds, service failure, retry	Branch/path testing
Security	SQL injection strings, brute-force rate limits, authorization bypass	Negative/security testing
Consistency	Timeout and retry; concurrent duplicate requests	Recovery, concurrency and idempotency

3.4 Coverage gaps and conclusion

A comprehensive strategy combines black-box ECP/BVA and state/decision-table testing with white-box statement, branch, condition and path coverage. It must verify both API responses and persistent transaction records. Security tests must be authorized and performed in a controlled environment. Defect logs should be mapped to the component, input class, branch and transaction state to identify recurring gaps.

4. Legacy Inventory Management: Stock Updates and Low-Stock Alerts

4.1 Inferred business logic

A typical stock update receives current stock, quantity change, reorder threshold and operation type. The system validates the quantity, applies the update atomically, stores the new stock and generates an alert when the stock crosses the configured threshold. Because the logs and exact rules are not attached, threshold values below are symbolic.

4.2 Functional units

Unit	Purpose
SKU lookup	Confirm product exists and is active
Quantity validation	Reject missing, non-numeric, zero where disallowed, or negative quantities
Stock calculation	Compute current stock plus receipt or minus issue
Constraint check	Prevent negative stock and integer overflow
Persistence	Commit the update atomically
Alert service	Trigger alert based on threshold rule
Audit/reconciliation	Record actor, old value, new value and transaction ID

4.3 Equivalence classes and boundary tests

Parameter	Equivalence classes	Boundary-focused tests
Quantity	Valid positive quantity; invalid zero/negative/non-numeric/too large	−1, 0, 1; maximum allowed and maximum + 1
Current stock	Zero; positive; corrupted/negative stored value	0, 1, configured maximum
New stock vs reorder level	Below; exactly equal; above threshold	T−1, T, T+1 where T is reorder level
SKU	Existing active; unknown; inactive	Known valid SKU; unknown SKU; deactivated SKU
Operation	Receipt; issue/dispatch; adjustment	Issue exactly available stock; issue one more than stock

4.4 Reconstructed test cases

TC	Initial state / operation	Expected result
INV01	Stock 20; receive 5	Stock becomes 25; no alert if threshold is below 25
INV02	Stock 10; issue 4	Stock becomes 6; alert only if 6 meets the defined threshold
INV03	Stock 5; threshold 5	Verify whether rule is ≤ threshold or < threshold; expected alert must match specification
INV04	Stock 6; threshold 5; issue 1	New stock 5; alert according to threshold rule
INV05	Stock 3; issue 4	Reject; stock remains 3; no partial update

The most important hidden boundary is whether the alert condition is stock < threshold or stock ≤ threshold. Logs around threshold−1, threshold and threshold+1 can reveal the rule. Tests must also check atomicity and concurrency, because a correct alert cannot compensate for an incorrect stock count.

5. Library Module: Resource Profiles and Path Testing

A compiled executable and CPU/memory profile do not reveal source-level control flow uniquely. A spike can suggest repeated processing, recursion, allocation, or a costly decision cluster, but profiling alone cannot prove which source construct caused it. The CFG below is therefore conceptual and should be validated with instrumentation, symbols or controlled experiments.

5.2 Profile-to-hypothesis mapping

Observed profile pattern	Possible cause	Test to confirm
CPU spike as record count grows	Nested scan, sorting, repeated filtering	Run 10, 100, 1,000, 10,000 records; compare runtime growth
Memory rises continuously	Unreleased buffers, retained result objects, cache growth	Repeated calls, heap snapshots, memory after cleanup
Spike only for certain file types	Expensive parser or malformed-file retry loop	Vary file type and malformed/valid samples
Long delay near permissions checks	Repeated authorization/database lookup	Vary user roles and record count; instrument call counts
CPU remains high after timeout	Unbounded retry or loop exit defect	Inject timeout; verify retry cap and cancellation

5.3 Resource-intensive path tests

Path	Test condition	Expected stability result
P1	Invalid request rejected before loading records	Low resource use; prompt error
P2	Valid request with zero records	Terminates; no unnecessary allocation
P3	One record, allowed	Correct result and cleanup
P4	Large set with all records denied	Bounded runtime; no result leak
P5	Large set with many allowed records	Correct pagination/limits; memory remains bounded
P6	Malformed record in middle of collection	Error isolated; processing terminates safely
P7	Repeated requests under stress	No progressive memory growth or thread exhaustion
P8	Cancellation or timeout during loop	Loop stops and resources are released

Measure CPU time, wall-clock latency, peak and retained memory, error rate, and correctness. Use a representative workload, a defined concurrency level and repeatable test data. Compare results against service-level limits. Apply profiling and sanitizers where available; test the executable only in an authorized, isolated environment.

Conclusion: path testing should target both ordinary and resource-heavy routes, especially loop exit, exceptional records, allocation and cleanup. Stability is demonstrated by bounded resource consumption and correct termination under load, not by code coverage alone.

6. Mobile Payment Gateway: State-Transition Testing

6.1 Inferred states and triggers

The named states are Pending, Success, Reversal and Timeout. A reasonable model treats Success, Reversal and Timeout as terminal outcomes for a single transaction attempt unless the API explicitly supports later reconciliation. A reversal generally compensates a previously successful debit; the actual contract must define whether reversal is allowed only after success or also during pending settlement.

6.2 Transition table

Current state	Event / trigger	Next state	Expected API / data behaviour
New	Valid request accepted	Pending	Unique transaction ID; amount/currency validated
Pending	Processor confirms approval	Success	One final success; ledger updated exactly once
Pending	Processor declines / compensation required	Reversal or failed outcome per contract	No unbalanced debit; reason recorded
Pending	Response deadline expires	Timeout	Status marked pending/timeout per contract; reconciliation possible
Success	Duplicate success callback	Success (idempotent)	No duplicate debit/credit

6.3 Test suite and data constraints

TC	Scenario	Expected result
PAY01	Valid amount/currency and accepted request	Pending with unique ID
PAY02	Processor approves while Pending	Success; exactly one ledger posting
PAY03	Processor declines	Defined failure/reversal response; no unexplained debit

The implicit technique is state-transition testing, supported by transition coverage and transition-pair coverage. Ensure every legal transition is exercised and each important illegal transition is rejected. Check both JSON response fields and backend ledger state; response-only testing is insufficient.

6.4 State model completeness

Because the actual JSON responses and transition triggers were not included, the exact API field names and status codes cannot be inferred. Validate the proposed model against the gateway contract, especially the semantics of Timeout and Reversal and the handling of late callbacks.

7. Encrypted Communication Module: Fuzzing and BVA

7.1 Decomposition

Component	Responsibilities	Tests
Input parser	Decode request and enforce format	Missing fields, invalid encoding, malformed structures
Data validator	Check lengths, ranges, types and allowed values	Minimum/maximum lengths, negative/overflow values
Encryption layer	Encrypt valid payload using approved parameters	Known test vectors, nonce/IV rules, key handling
Output encoder	Encode ciphertext and metadata	Output format, truncation, invalid characters
Error handler	Return safe errors and clean up resources	Malformed input, allocation failure, no sensitive leakage

7.2 BVA and fuzzing plan

Field	Boundary/invalid inputs	Expected safe behaviour
Message length (limit L)	0, 1, L−1, L, L+1, very large length	Accept only documented range; reject safely beyond L
Key identifier	Minimum/maximum length; unknown ID; empty	Reject invalid ID; never disclose key material
Nonce/IV	Correct length; one byte short/long; repeated nonce in test environment	Reject invalid length; enforce cryptographic requirements
Encoding	Valid encoding; invalid byte sequence; embedded NUL	Reject or normalize only as specified; no truncation confusion

7.3 Example test cases

TC	Input	Expected result
CR01	Small valid payload within limit	Correctly encoded encrypted output; decrypt/verify in test harness
CR02	Payload length exactly L	Accepted if L is inclusive limit
CR03	Payload length L+1	Rejected with controlled error
CR04	Empty required field	Validation error; encryption not invoked

Fuzzing should use a controlled test harness and a corpus of valid and malformed inputs. Apply strict length checks before allocation/copying, safe integer arithmetic, bounds-checked parsing, authenticated encryption, and generic external error messages. Boundary tests and fuzzing can reveal potential overflow or injection defects, but they do not by themselves prove the absence of vulnerabilities. Run only against software you are authorized to test.

8. Insurance Premium Calculator: Statement vs Branch/Path Coverage

100% statement coverage means every statement ran at least once. It does not guarantee that each decision outcome or each atomic condition was tested. A test can execute the premium calculation statement while never testing important combinations of driver age, claim history and vehicle category.

8.1 Illustrative decision logic and CFG

Because the actual source code is not supplied, use this illustrative rule set: if age is below 25, apply a young-driver surcharge; if claim history is true, apply a claim surcharge; otherwise retain the base premium. A production premium model may contain additional factors.

The illustrative CFG has two binary decision nodes, so V(G) = 2 + 1 = 3 independent paths.

Path	Age < 25?	Claim flag?	Expected calculation
P1	False	False	Base premium only
P2	False	True	Base + claim surcharge
P3	True	True or False	Base + young-driver surcharge, plus claim surcharge if claim flag is true

8.2 Reconstructed test suite

TC	Age condition	Claim history	Expected branch / result
INS01	Age = 25 (not below 25)	False	Base premium; both decisions False
INS02	Age = 25	True	Claim surcharge only
INS03	Age = 24	False	Young-driver surcharge only
INS04	Age = 24	True	Both surcharges
INS05	Age at lower valid limit	False	Valid input boundary
INS06	Age at upper valid limit	True	Valid input boundary
INS07	Age below minimum / above maximum	Any	Reject invalid age; do not calculate
INS08	Claim flag missing or invalid	Invalid	Validation error

8.3 Coverage evaluation

Statement coverage: execute all calculation and output statements.

Branch coverage: test True and False outcomes of each if condition.

Condition coverage: if a decision contains AND/OR, make each atomic condition independently True and False.

Multiple-condition coverage: test relevant combinations, especially when one condition can mask another.

Path coverage: cover each feasible independent path; exhaustive path coverage may be impossible when loops or many conditions exist.

A claim of 100% branch and path coverage should be supported by the actual CFG and coverage report. For the illustrative acyclic two-decision example, all four combinations cover both decisions' outcomes and reduce the risk of missed surcharges or incorrect totals.

9. Cloud File Synchronization: Lost-File Defects

9.1 Likely equivalence partitions

Dimension	Equivalence classes to investigate	Boundary / stress points
File size	Empty, small, typical, near maximum, oversized	0 bytes, 1 byte, limit−1, limit, limit+1
File type	Supported, unsupported, extension/content mismatch, special characters	Large metadata, Unicode name, very long path
Network	Stable/fast, slow, intermittent, disconnected	High latency, packet loss, disconnect during upload/download
File state	New, modified, renamed, deleted, conflict	Edit during sync; delete while offline; simultaneous edits
Storage	Enough space, nearly full, full	Available space below/equal/above required size
Concurrency	Single device, two devices, many devices	Same file changed concurrently; repeated retries

9.3 Test cases integrating BVA and stress

TC	Scenario	Expected result
SYNC01	Empty file and 1-byte file	Both synchronize correctly if supported
SYNC02	File size at limit−1, limit, limit+1	Boundary follows documented maximum; oversized file rejected safely
SYNC03	Large file over slow network	Progress/resume works; no corrupted final file
SYNC04	Network drops halfway through transfer	Resume or restart without data loss; source remains intact
SYNC05	Checksum mismatch after transfer	Do not mark synchronized; retry or report integrity failure
SYNC06	Unsupported file type	Clear error or documented exclusion; never silently delete source

9.4 Why this strategy prevents data loss

ECP focuses on representative file, network and state classes; BVA targets file-size and storage thresholds; stress testing exposes queue, memory and concurrency issues. Add checksums, version IDs, atomic commit, resumable transfers, retry limits, conflict handling and audit logs. Verify that a file is marked synchronized only after integrity checks and server acknowledgement. Actual lost-file logs are not included, so the specific original ECP cannot be established from the supplied prompt alone.

10. E-commerce Search and Filter: Decision-Table Testing

10.1 Inferred filter logic

The query parameters and returned datasets are intended to reveal how filters combine. A common model uses AND between different filter categories (e.g., category and price) and OR within a multi-select category (e.g., brand A or brand B). NOT excludes matching records. The actual SQL queries were not attached, so this is a model to validate against observed queries/results.

Example predicates: C = category matches; P = price is in range; B = selected brand matches; N = item is explicitly excluded. One possible rule is Result = C AND P AND B AND NOT N. If no brand is selected, the brand predicate may be omitted rather than treated as false; this rule must be confirmed.

10.2 Decision table

Rule	Category match (C)	Price match (P)	Brand match (B)	Excluded (N)	Expected result
R1	Y	Y	Y	N	Include product
R2	N	Y	Y	N	Exclude: category mismatch
R3	Y	N	Y	N	Exclude: price mismatch
R4	Y	Y	N	N	Exclude: brand mismatch

10.3 Test combinations

TC	Search/filter input	Expected validation
ECOM01	Category only	All returned items belong to selected category
ECOM02	Minimum price = maximum price	Verify inclusive/exclusive range contract
ECOM03	Price just below, at and just above each bound	Boundary results match documented inclusivity
ECOM04	Two brands selected	OR/IN behaviour returns either selected brand
ECOM05	Category + price + brand	AND across filter categories if that is the specified rule
ECOM06	Apply NOT/excluded term	Excluded products are absent

10.4 Model completeness and adequacy

A full truth table for four Boolean predicates has $2^4 = 16$ combinations. The compact decision table above lists the highest-value rules and representative invalid cases; it is not an exhaustive 16-row truth table. To claim complete logical combination coverage, enumerate all 16 combinations or use modified condition/decision coverage where appropriate, then add classes for omitted filters, multiple selected values, invalid inputs, sorting and pagination.

Validate the reconstructed logic by comparing each test's input parameters with the generated SQL and returned product IDs. Use parameterized queries, allow-listed sort fields, server-side validation and consistent pagination. Do not infer exact AND/OR semantics from output alone when several data sets could produce the same result; use carefully chosen records that distinguish competing hypotheses.

Code of Conduct:

*I **BHUVANESHWARI B A (192421195)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: BHUVANESHWARI B A (192421195)

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation